Pandas 详细学习讲义

从理解表格到完成数据分析题

数据科学导论复习配套

基础操作、分组统计、分箱、合并与重塑

本讲义面向已经初步学习 NumPy、准备学习 Pandas 的读者。每一部分尽量同时回答：这个功能是什么、解决什么问题、参数怎么填、代码怎么读、输出是什么，以及容易在哪里写错。

全文以学生成绩和商品销售为具体案例。示例数据均为教学用数据，可直接复制运行。文末提供按需求查找的函数索引，以及综合练习和答案。

内容覆盖 Pandas 基础与常见的数据探索操作；暂不包含 Matplotlib、Seaborn 等绘图内容。本讲义用于复习，不代表正式考试范围公告。

2026 年 10 月

目录

1 Pandas 到底是什么，能做什么 **6**

1.1 Pandas 是处理表格的工具库 . . . 6

1.2 Pandas 与 NumPy 的关系 . . . 6

1.3 导入 . . . 6

2 先理解字典，再理解表格 **7**

2.1 Python 字典是什么 . . . 7

2.2 字典的值可以是列表 . . . 7

2.3 列表、字典与 Pandas 对象的区别 . . . 7

3 核心对象：DataFrame、Series 和 Index **8**

3.1 DataFrame：带标签的二维表格 . . . 8

3.2 Series：带索引的一维数据 . . . 9

3.3 Index：标签不等于位置 . . . 9

3.4 Series 与 NumPy 数组的区别 . . . 10

4 创建 Series 和 DataFrame **10**

4.1 用列表创建 Series . . . 10

4.2 用字典创建 Series . . . 10

4.3 用“列名对应列表”的字典创建 DataFrame . . . 11

4.4 用二维列表创建 DataFrame . . . 11

4.5 用“每行一个字典”的列表创建 . . . 11

4.6 用 Series 创建 DataFrame . . . 11

5 全文案例：商品销售表 **12**

6 读取与保存文件 **12**

6.1 read_csv()：读取 CSV . . . 12

6.2 read_excel()：读取 Excel . . . 13

6.3 路径是什么 . . . 13

6.4 to_csv() 与 to_excel() . . . 14

7 查看表格：先知道自己拿到什么 **14**

7.1 head() 与 tail() . . . 14

7.2 shape、size、len、columns、index、dtypes . . . 14

7.3 info()：结构和缺失情况 . . . 15
7.4 describe()：描述统计表 . . . 15
7.5 转成数组或列表 . . . 16

8 选列：一层和两层方括号 16

8.1 一列 Series . . . 16
8.2 一列 DataFrame . . . 16
8.3 多列 DataFrame . . . 16

9 iloc：按照位置选行列 17

9.1 基本结构 . . . 17
9.2 选行 . . . 17
9.3 同时选行列 . . . 17
9.4 取单元格与保留二维表格 . . . 18

10 loc：按照标签选行列 18

10.1 基本结构 . . . 18
10.2 选择指定标签 . . . 18
10.3 标签切片包含右端 . . . 18
10.4 列标签也能切片 . . . 19
10.5 单元格 . . . 19
10.6 loc 与 iloc 的核心对照 . . . 19
10.7 at 与 iat：专门选单个值 . . . 20

11 条件筛选：找到需要的记录 20

11.1 单条件选行 . . . 20
11.2 筛选行并指定输出列 . . . 20
11.3 多个条件 . . . 20
11.4 isin()：是否属于给定集合 . . . 21
11.5 between()：区间筛选 . . . 21
11.6 query()：可选的字符串表达式 . . . 22
11.7 布尔 Series 给 loc，位置数组给 iloc . . . 22

12 字符串筛选与日期 22

12.1 字符串列使用.str . . . 22
12.2 contains()：包含某段文字 . . . 22

12.3 startswith() 与 endswith() . . . 23
12.4 转换日期 . . . 23

13 统计：一列、全表和不同值 23

13.1 一列的统计方法 . . . 23
13.2 axis：逐列与逐行统计 . . . 24
13.3 count() 与 size . . . 24
13.4 unique()、nunique()、value_counts() . . . 24
13.5 布尔计数 . . . 25

14 新增、修改和删除 25

14.1 新增一列 . . . 25
14.2 assign()：返回带新列的表格 . . . 26
14.3 insert()：指定列位置 . . . 26
14.4 条件修改：loc 赋值 . . . 26
14.5 np.where() 生成条件列 . . . 26
14.6 drop()：删除行或列 . . . 27
14.7 del 和 pop：直接删除列 . . . 27
14.8 rename()：修改标签名称 . . . 27
14.9 replace()：替换数据值 . . . 28

15 索引管理：set_index、reset_index、reindex 28

15.1 set_index()：把一列变成行索引 . . . 28
15.2 reset_index()：恢复默认整数索引 . . . 28
15.3 reindex()：按指定标签重新排列或补出标签 . . . 28
15.4 整体替换标签 . . . 29

16 排序：按值、按索引、多键排序 30

16.1 sort_values()：按列值排序 . . . 30
16.2 多键排序 . . . 30
16.3 sort_index()：按标签排序 . . . 30
16.4 排序后取前几条 . . . 30
16.5 nlargest()：取数值最大的若干条 . . . 30

17 缺失值、重复值和类型转换 31

17.1 isna() 与 notna() . . . 31

17.2 dropna()：删除缺失记录 . . . 31
17.3 fillna()：填充缺失值 . . . 31
17.4 duplicated() 与 drop_duplicates() . . . 32
17.5 astype() 与 to_numeric() . . . 32

18 copy：保留原表，避免链式赋值 33
18.1 赋值不是复制 . . . 33
18.2 copy() 创建独立表格副本 . . . 33
18.3 筛选后要修改，显式 copy . . . 33

19 groupby：分组统计 33
19.1 groupby 到底做什么 . . . 33
19.2 分组均值 . . . 34
19.3 as_index=False：组别保持普通列 . . . 34
19.4 agg()：一次计算多个统计量 . . . 34
19.5 命名聚合：直接指定输出列名 . . . 35
19.6 size() 与 count() 的差别 . . . 35
19.7 多个字段分组 . . . 35
19.8 transform()：把组统计结果放回每一行 . . . 36

20 cut：把连续数值分成类别 36
20.1 分箱是什么 . . . 36
20.2 pd.cut()：指定边界 . . . 36
20.3 默认 right=True . . . 37
20.4 统计各年龄组数量 . . . 37
20.5 年龄组中的男女数量与比例 . . . 37
20.6 qcut()：按分位数分组 . . . 38

21 merge：连接两张表 38
21.1 为什么要合并表格 . . . 38
21.2 left merge：保留左表全部记录 . . . 38
21.3 常见连接方式 . . . 39
21.4 两边连接列名称不同 . . . 39
21.5 多个连接键 . . . 39
21.6 为什么合并后行数会增加 . . . 40

22 concat：把表格上下或左右拼接 **40**

22.1 上下追加记录 40

22.2 左右拼接列 40

23 透视表、交叉表与长宽表转换 **41**

23.1 pivot_table()：二维分组汇总 41

23.2 pivot()：只重排，不聚合 42

23.3 crosstab()：分类交叉计数 42

23.4 melt()：宽表变成长表 42

23.5 stack() 与 unstack() 43

24 综合案例一：销售分析流水线 **43**

24.1 题意 43

24.2 分步答案 44

24.3 为什么这个顺序重要 44

25 综合案例二：公司投资统计与合并 **45**

25.1 准备数据 45

25.2 任务 45

26 按需求查找：常用函数速查 **46**

27 综合练习与参考答案 **47**

27.1 练习一：标签、位置和输出类型 47

27.2 练习二：多条件筛选与排序 47

27.3 练习三：缺失值与组内填补 47

27.4 练习四：分组汇总 48

27.5 练习五：分箱边界 48

27.6 练习六：合并并核对匹配情况 48

28 做题前的最后检查 **49**

1 Pandas 到底是什么，能做什么

1.1 Pandas 是处理表格的工具库

Pandas 是 Python 中用于处理表格数据的工具库。可以把它理解成“通过代码操作 Excel 表格”。

例如，有一张学生表：

姓名	班级	成绩
Tom	A	80
Amy	B	90
Bob	A	55

使用 Pandas 可以：

- 读取 Excel 或 CSV 文件。
- 找出不及格的学生。
- 计算全体学生或每个班的平均成绩。
- 按成绩排序。
- 添加“是否及格”一列。
- 处理缺失或重复记录。
- 把学生表和另一张信息表合并。
- 将处理后的结果保存到文件。

关键点：学习 Pandas 不是记住一堆孤立函数，而是把需求转成表格操作：读入、查看、筛选、修改、统计、排序、合并和保存。

1.2 Pandas 与 NumPy 的关系

NumPy 擅长对数组中的数进行计算。Pandas 在数值计算之外，提供行索引、列名、不同列的数据类型和各种表格分析功能。

普通 NumPy 数组一般使用统一的 dtype；Pandas 表格可以让姓名列存字符串，成绩列存整数，日期列存日期。

1.3 导入

```
import pandas as pd
import numpy as np
```

pd 是 Pandas 的简称，np 是 NumPy 的简称。

本讲义中使用 NumPy 的地方主要包括：创建数值数据、表示缺失值、条件生成新列等。

2 先理解字典，再理解表格

2.1 Python 字典是什么

字典使用大括号 \{\}，保存“键与对应值”。

```
info = {
    "name": "Tom",
    "age": 18
}
```

- "name" 是键，对应的值是 "Tom"。
- "age" 是键，对应的值是 18。
- 冒号表示对应关系，各项用逗号分开。

键通常可理解成某项信息的名字。这里的 "name" 不是特殊命令，只是字符串。

可以通过键读取值：

```
info["name"]
```

结果是字符串 "Tom"。

2.2 字典的值可以是列表

```
data = {
    "name": ["Tom", "Amy", "Bob"],
    "score": [80, 90, 55]
}
```

这个字典保存两组对应关系：

- "name" 对应三个人的姓名列表。
- "score" 对应三个人的成绩列表。

此时 data 是字典，还不是 Pandas 表格。

2.3 列表、字典与 Pandas 对象的区别

```
a = np.array([1, 2, 3])
```

这里列表是输入，创建出的 a 是 NumPy 数组。

```
df = pd.DataFrame(data)
```

这里字典是输入，创建出的 df 是 Pandas 表格。

关键点：Pandas 不是字典，NumPy 不是列表。它们是工具库；列表和字典只是创建数据对象时的输入材料。

括号的作用：

形式	在这些代码中的含义
`[1, 2, 3]`	创建列表
`("a", "b")`	创建元组
`\{"name": ...\}`	创建字典
`pd.DataFrame(...)`	调用创建表格的函数接口
`df["score"]`	对已有对象进行索引或选择

同一种括号可能在不同位置承担不同功能。例如方括号既可创建列表，也可用于数据选择。

3 核心对象：DataFrame、Series 和 Index

3.1 DataFrame：带标签的二维表格

```
students = pd.DataFrame({
    "name": ["Tom", "Amy", "Bob"],
    "class": ["A", "B", "A"],
    "score": [80, 90, 55]
})
print(students)
```

输出：

```
  name class  score
0  Tom     A     80
1  Amy     B     90
2  Bob     A     55
```

- `name`、`class`、`score` 是列名。
- 左侧的 0、1、2 是行索引。
- 每行是一条完整记录。
- 每列描述一个属性或特征。

从字典中的普通列表创建时，各列列表长度必须相同，才能逐行对应。

3.2 Series：带索引的一维数据

从表格选择一列：

```
scores = students["score"]
print(scores)
```

输出：

```
0    80
1    90
2    55
Name: score, dtype: int64
```

右边是成绩，左边是每个成绩对应的索引。

Series 也可以独立创建：

```
s = pd.Series(
    [80, 90, 55],
    index=["Tom", "Amy", "Bob"],
    name="score"
)
```

index 指定标签，name 指定这个 Series 的名字。

3.3 Index：标签不等于位置

创建自定义行标签：

```
students = pd.DataFrame({
    "name": ["Tom", "Amy", "Bob"],
    "score": [80, 90, 55]
}, index=[10, 20, 30])
```

输出：

```
   name  score
10  Tom     80
20  Amy     90
30  Bob     55
```

记录	位置	行标签
Tom	0	10
Amy	1	20
Bob	2	30

关键点：位置始终从 0 开始；标签是给记录起的标记，可以是整数、字符串或日期。标签为 20 的行不代表它位于第 21 行。

3.4 Series 与 NumPy 数组的区别

Series 除了值，还保存索引标签。两个 Series 运算时，通常按标签对齐。

```
a = pd.Series([10, 20], index=["x", "y"])
b = pd.Series([1, 2], index=["y", "x"])

print(a + b)
```

输出：

```
x    12
y    21
dtype: int64
```

标签 x 的 10 与标签 x 的 2 相加，标签 y 的 20 与标签 y 的 1 相加。

易错点：不要总把 Series 运算理解成“第一个加第一个”。NumPy 普通数组主要按位置计算；Series 通常按标签对齐。

4 创建 Series 和 DataFrame

4.1 用列表创建 Series

```
s1 = pd.Series([4, 7, -5, 3])

s2 = pd.Series(
    [4, 7, -5, 3],
    index=["d", "b", "a", "c"]
)
```

不指定索引时，默认使用 0、1、2、3。

4.2 用字典创建 Series

```
s = pd.Series({
    "Ohio": 35000,
    "Texas": 71000,
    "Oregon": 16000
})
```

字典键变成索引，字典值变成数据值。

4.3 用“列名对应列表”的字典创建 DataFrame

```
products = pd.DataFrame({
    "name": ["Apple", "Banana"],
    "price": [5, 3]
})
```

键变成列名，列表提供列数据。

4.4 用二维列表创建 DataFrame

```
products = pd.DataFrame(
    [["Apple", 5], ["Banana", 3]],
    columns=["name", "price"]
)
```

每个内部列表提供一行，`columns` 指定列名。

4.5 用“每行一个字典”的列表创建

```
products = pd.DataFrame([
    {"name": "Apple", "price": 5},
    {"name": "Banana", "price": 3}
])
```

每个字典描述一条记录。某行缺少字段时，相应位置可以出现缺失值。

4.6 用 Series 创建 DataFrame

```
names = pd.Series(
    ["Tom", "Amy"],
    index=["r1", "r2"]
)

scores = pd.Series(
    [80, 90],
```

```
    index=["r1", "r2"]
)

students = pd.DataFrame({
    "name": names,
    "score": scores
})
```

Series 按索引对齐。若索引不完全相同，结果可能出现缺失值。

把一个 Series 变成单列表格：

```
score_table = scores.to_frame(name="score")
```

5 全文案例：商品销售表

为了让后面的代码能独立运行，先统一准备一张案例表：

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Apple", "Pear", "Banana", "Orange", "Kiwi"],
    "group": ["A", "B", "A", "B", "A"],
    "price": [10, 20, 15, 12, 8],
    "qty": [2, 1, 4, 3, 5],
    "score": [80.0, 90.0, np.nan, 70.0, 85.0],
    "year": [1935, 1944, 1948, 1952, 1954],
    "city": ["Beijing", "Shanghai", "Beijing", "Shenzhen", None]
}, index=[10, 20, 30, 40, 50])
```

- 每行是一条商品记录。
- 行标签是 10、20、30、40、50。
- 行位置是 0、1、2、3、4。
- `price` 是单价，`qty` 是数量。
- `score` 和 `city` 各有一个缺失值。

关键点：后文写 df 时，默认指这里的案例表。尝试修改类操作前，可重新运行创建代码恢复原表。

6 读取与保存文件

6.1 read_csv()：读取 CSV

```
df = pd.read_csv("retail_quiz.csv")
```

返回值是 DataFrame。

常见参数：

- 第一个参数：路径或文件名。
- `nrows=40`：只读取前 40 行数据。
- `index_col="line_id"`：把指定列作为行索引。
- `header=None`：文件没有表头时使用。
- `encoding=...`：按文件实际编码读取。

```
first_40 = pd.read_csv("investment.csv", nrows=40)

indexed = pd.read_csv(
    "retail_quiz.csv",
    index_col="line_id"
)
```

易错点：正常 CSV 有列名时，不要随意加 header=None。否则原来的表头会被当成第一条数据。

6.2 read_excel()：读取 Excel

```
df = pd.read_excel("admission.xlsx", sheet_name=0)
```

`sheet_name=0` 表示第一张工作表。也可以指定工作表名称：

```
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

`header=None` 表示没有表头。

易错点：读取工作表用 `sheet_name`，不要照抄错误写法 `DataFrame=0`。

6.3 路径是什么

只写文件名时，按 Python 当前工作目录寻找文件。不一定是 Notebook 文件所在目录。

查看当前工作目录：

```
import os
print(os.getcwd())
```

Windows 完整路径可使用原始字符串：

```
df = pd.read_csv(
    r"C:\Users\LENOVO\Desktop\data.csv"
```

```
)
```

6.4 to_csv() 与 to_excel()

```
df.to_csv(
    "result.csv",
    index=False,
    encoding="utf-8-sig"
)

df.to_excel(
    "result.xlsx",
    index=False,
    sheet_name="Result"
)
```

`index=False` 表示不额外输出行索引列。若索引本身有业务意义，就按题意保留或先恢复成普通列。

关键点：同名文件会被覆盖。练习时使用独立的练习文件名。

7 查看表格：先知道自己拿到什么

7.1 head() 与 tail()

```
df.head()
df.head(2)
df.tail(1)
```

- `head()`：默认前 5 行。
- `head(2)`：前 2 行。
- `tail(1)`：最后 1 行。

这些操作不修改原表。

7.2 shape、size、len、columns、index、dtypes

使用第 5 节的案例表：

```
df.shape
df.size
len(df)
df.columns
```

```
df.index
df.dtypes
```

写法	含义	案例结果
`df.shape`	行数、列数	`(5, 7)`
`df.size`	总单元格数	`35`
`len(df)`	行数	`5`
`df.columns`	列标签	name、group 等
`df.index`	行标签	10、20、30、40、50
`df.dtypes`	每列的数据类型	一个 Series

关键点：这些属性不加括号。特别注意 `dtypes` 有 s，因为表格可能每列一种类型；Series 的元素类型通常查看 `s.dtype`。

7.3 info()：结构和缺失情况

```
df.info()
```

会直接打印：行数、列名、每列非缺失数量、类型等信息。

易错点：`info()` 返回 None。不要写 `summary = df.info()`，然后期待 summary 是统计表。

7.4 describe()：描述统计表

```
df.describe()
```

默认主要汇总数值列：

字段	含义
count	非缺失值数量
mean	平均值
std	标准差
min	最小值
25%	第一四分位数
50%	中位数
75%	第三四分位数
max	最大值

包含非数值列：

```
df.describe(include="all")
```

字符串列可能显示不同值数量、最常见值及其频数。

7.5 转成数组或列表

```
arr = df.to_numpy()
values = df["price"].to_numpy()
price_list = df["price"].to_list()
```

- `to_numpy()` 去掉行列标签，获取数组表示。
- Series 的 `to_list()` 得到普通 Python 列表。
- 混合类型表格转换成一个数组时，可能得到 object dtype。

8 选列：一层和两层方括号

8.1 一列 Series

```
prices = df["price"]
```

返回 Series，带行索引和一组价格。

8.2 一列 DataFrame

```
price_table = df[["price"]]
```

返回只有一列的 DataFrame。

这里外层方括号执行选择，内层列表 `["price"]` 是需要选择的列名列表。

8.3 多列 DataFrame

```
selected = df[["name", "price", "qty"]]
```

返回三列表格，列顺序按照列表中的顺序。

写法	返回类型
`df["price"]`	Series
`df[["price"]]`	DataFrame
`df[["price", "qty"]]`	DataFrame

易错点：普通单层列索引不要写 `df["price", "qty"]` 来选两列。应该传入列表 `df[["price", "qty"]]`。

9 iloc：按照位置选行列

9.1 基本结构

```
df.iloc[row_selection, column_selection]
```

逗号左边控制行，右边控制列。位置从 0 开始，切片右端不包含。

9.2 选行

```
df.iloc[0]
df.iloc[:2]
df.iloc[1:3]
df.iloc[[0, 2, 4]]
```

分别表示：

- 第一行，返回 Series。
- 前两行，返回 DataFrame。
- 第二、第三行。
- 第一、第三、第五行。

案例表中：

```
df.iloc[1:3]
```

选到 Pear 和 Banana，标签为 20、30。

9.3 同时选行列

```
df.iloc[:3, :2]
df.iloc[[0, 2], [0, 2]]
df.iloc[:, 2]
```

分别是：

- 前三行、前两列。
- 行位置 0、2，列位置 0、2 的交叉表格。
- 所有行中的第三列。

关键点：DataFrame 的 iloc 同时传行列列表时，选取行列交叉形成的表格。不要套用 NumPy 的两个整数数组配对取坐标规则。

9.4 取单元格与保留二维表格

```
df.iloc[0, 2]
df.iloc[[0], [2]]
```

第一个得到标量 10；第二个得到 1 行、1 列的 DataFrame。

易错点：单个整数通常会降低结果维度；列表或切片通常保留表格结构。考试要求结果为 DataFrame 时，要检查返回类型。

10 loc：按照标签选行列

10.1 基本结构

```
df.loc[row_labels, column_labels]
```

loc 按行标签和列标签选择，不是按位置。

10.2 选择指定标签

```
df.loc[20]
df.loc[[10, 30]]
df.loc[[10, 30], ["name", "price"]]
```

最后一句得到：

```
      name  price
10   Apple     10
30  Banana     15
```

10.3 标签切片包含右端

```
df.loc[20:40, ["name", "price"]]
```

得到标签 20、30、40：

```
      name  price
20    Pear     20
30  Banana     15
40  Orange     12
```

这里的标签索引已经按升序排列。无序索引或重复标签的切片可能有额外限制，不要把 loc 机械理解成按数值进行区间过滤。

10.4 列标签也能切片

```
df.loc[:, "name":"qty"]
```

按当前列顺序选取 name 到 qty，包括中间的 group、price，并包含 qty。

关键点：列切片依赖当前列顺序，不是按字母顺序决定范围。

10.5 单元格

```
df.loc[20, "price"]
```

得到 20。

10.6 loc 与 iloc 的核心对照

写法	根据什么选	切片右端
`df.iloc[1:3]`	行位置	不包含位置 3
`df.loc[20:40]`	行标签	标签切片包含 40

再看一个非连续索引例子：

```
x = pd.DataFrame({
    "value": [100, 200, 300, 400]
}, index=[1, 3, 5, 7])

print(x.iloc[1:3])
print(x.loc[1:3])
```

输出：

```
   value
3    200
5    300

   value
1    100
3    200
```

易错点：行标签不存在可能报 KeyError，位置越界可能报 IndexError。这些是运行时索引错误，不一定是 Python 语法错误。

10.7 at 与 iat：专门选单个值

```
df.at[20, "price"]
df.iat[1, 2]
```

二者都得到 20：

- at 按标签。
- iat 按位置。

多行、多列选择使用 loc、iloc。

11 条件筛选：找到需要的记录

11.1 单条件选行

解决的问题：找出价格大于 10 的商品。

```
mask = df["price"] > 10
selected = df[mask]
```

也可以写：

```
selected = df[df["price"] > 10]
```

得到 Pear、Banana、Orange。

条件本身是带索引的布尔 Series。表格用它选择 True 对应的行。

11.2 筛选行并指定输出列

```
selected = df.loc[
    df["price"] > 10,
    ["name", "price"]
]
```

这是常用结构：

```
df.loc[condition, output_columns]
```

11.3 多个条件

解决的问题：价格至少 10，并且数量至少 3。

```
selected = df.loc[
    (df["price"] >= 10) & (df["qty"] >= 3),
```

```
    ["name", "price", "qty"]
]
```

结果：

```
      name  price  qty
30  Banana     15    4
40  Orange     12    3
```

- `&`：并且。
- `|`：或者。
- `~`：对布尔条件取反。

```
df.loc[
    (df["group"] == "B") | (df["qty"] == 5),
    ["name", "group"]
]
```

选到 Pear、Orange、Kiwi。

易错点：每个比较条件加括号。数组和 Series 条件不要直接使用 Python 的 and、or，也不要写 `10 <= df["price"] <= 20`。

11.4 isin()：是否属于给定集合

```
selected = df[
    df["name"].isin(["Apple", "Kiwi"])
]
```

选出名称是 Apple 或 Kiwi 的记录。

不属于集合：

```
selected = df[
    ~df["name"].isin(["Apple", "Kiwi"])
]
```

易错点：isin 的参数是一个候选集合或列表，不是单独的字符串。应写 `isin(["Apple"])`。

11.5 between()：区间筛选

```
df[df["price"].between(10, 15)]
```

默认包含两端，选出价格 10、12、15。

指定左闭右开：

```
df[df["price"].between(10, 15, inclusive="left")]
```

选价格满足 $10 \le p < 15$ 的记录。

11.6 query()：可选的字符串表达式

```
threshold = 10
result = df.query("price >= @threshold and qty >= 3")
```

query 内是表达式字符串。引用外部变量时加 `@`。

初学阶段优先掌握 `loc[mask, columns]`，不必为了简短强行使用 query。

11.7 布尔 Series 给 loc，位置数组给 iloc

```
mask = df["qty"] >= 4

df.loc[mask, ["name", "qty"]]
df.iloc[mask.to_numpy(), [0, 3]]
```

两个结果都选到 Banana、Kiwi。

关键点：loc 接受带标签的布尔 Series，并按标签对齐。iloc 需要位置型选择；用布尔数组时，应确保它对应当前行顺序。不要直接把带标签的布尔 Series 交给 iloc。

12 字符串筛选与日期

12.1 字符串列使用.str

```
df["name"].str.len()
df["name"].str.upper()
df["name"].str.lower()
```

分别计算字符串长度、转大写、转小写。

案例中名称长度为：

```
[5, 4, 6, 6, 4]
```

12.2 contains()：包含某段文字

```
selected = df.loc[
    df["name"].str.contains("an", regex=False, na=False),
    ["name"]
]
```

选到 Banana、Orange。

- `regex=False`：把搜索内容当普通文本。
- `na=False`：缺失值视为不匹配。

contains 默认支持正则表达式，查普通文字时写 regex=False 更明确。

12.3 startswith() 与 endswith()

```
df[df["city"].str.startswith("Bei", na=False)]
df[df["name"].str.endswith("e", na=False)]
```

第一句选出城市以 Bei 开头的记录；第二句选出名称以 e 结尾的记录。

12.4 转换日期

```
events = pd.DataFrame({
    "date": ["2026-10-01", "2026-10-12"],
    "sales": [100, 150]
})

events["date"] = pd.to_datetime(events["date"])
events["year"] = events["date"].dt.year
events["month"] = events["date"].dt.month
```

字符串转换成日期后，使用 `.dt` 访问日期组成。

易错点：字符串列的工具是.str；日期列的工具是.dt；不要在普通整数列上直接调用字符串方法。

13 统计：一列、全表和不同值

13.1 一列的统计方法

```
df["price"].sum()
df["price"].mean()
df["price"].median()
df["price"].min()
df["price"].max()
df["price"].std()
```

前五项结果：

```
65
```

```
13.0
12.0
8
20
```

Pandas 的 Series 标准差默认使用 `ddof=1`。想与 NumPy 默认口径一致，应写：

```
df["price"].std(ddof=0)
```

13.2 axis：逐列与逐行统计

```
scores = pd.DataFrame({
    "math": [80, 60],
    "english": [90, 80],
    "chinese": [70, 100]
})

scores.mean(axis=0)
scores.mean(axis=1)
```

- `axis=0`：每列一个平均值，结果是 70、85、85。
- `axis=1`：每行一个平均值，结果是 80、80。

这与 NumPy 的轴方向一致。

易错点：混合姓名、城市等字符串列的表格，不要不加判断地对整张表求平均。先选择所需数值列，例如 `df[["price", "qty"]].mean()`。

13.3 count() 与 size

```
df["score"].count()
df["score"].size
```

结果分别是 4、5：

- count：非缺失值数量。
- size：元素数量，包括缺失。

13.4 unique()、nunique()、value_counts()

```
df["group"].unique()
df["group"].nunique()
df["group"].value_counts()
```

分别得到：

```
array(['A', 'B'])
2

group
A    3
B    2
```

- unique：有哪些不同值。
- nunique：不同值有几个。
- value_counts：每个值出现多少次。

Series.unique 通常保持首次出现的顺序，不要认为它和 np.unique 一样默认排序。

统计比例：

```
df["group"].value_counts(normalize=True)
```

A 为 0.6，B 为 0.4。

默认计数不包含缺失；要包含缺失：

```
df["city"].value_counts(dropna=False)
```

13.5 布尔计数

```
n_large = (df["qty"] >= 3).sum()
```

得到 3，因为 Banana、Orange、Kiwi 满足条件。

关键点：记录数不一定等于人数或销量。若表格某列已经记录人数，应对那列 sum；len 或布尔求和只统计记录条数。

14 新增、修改和删除

14.1 新增一列

解决的问题：计算每条记录的销售额。

```
x = df.copy()
x["revenue"] = x["price"] * x["qty"]
```

新列值：

```
[20, 20, 60, 36, 40]
```

新列默认添加在末尾。给已有列名赋值，会覆盖该列。

给每行相同值：

```
x["country"] = "China"
```

给列表时，长度要与行数相同：

```
x["flag"] = [1, 0, 1, 0, 1]
```

14.2 assign()：返回带新列的表格

```
x = df.assign(revenue=df["price"] * df["qty"])
```

返回新 DataFrame，原来的 df 不变。适合串联操作。

14.3 insert()：指定列位置

```
x = df.copy()
x.insert(3, "revenue", x["price"] * x["qty"])
```

在列位置 3 之前插入，即放在原来的 price 后、qty 前。

关键点：DataFrame.insert 原地修改并返回 None。不要写 `x = x.insert(...)`。它与 NumPy 的 np.insert 返回新数组不同。

14.4 条件修改：loc 赋值

解决的问题：把 B 组商品的价格统一改成 99。

```
x = df.copy()
x.loc[x["group"] == "B", "price"] = 99
```

必须在同一个 loc 里指定行条件和目标列。

易错点：不要依赖 `x[x["group"] == "B"]["price"] = 99` 这种链式赋值。它可能无法修改目标表格，并且不同版本行为有差异。

14.5 np.where() 生成条件列

```
x = df.copy()
x["large_order"] = np.where(x["qty"] >= 3, 1, 0)
```

新列是 0、0、1、1、1。

这里调用的是 NumPy 的 where，条件和原始数据来自 Pandas 的列。

14.6 drop()：删除行或列

删除列：

```
x = df.drop(columns=["city", "year"])
```

删除行标签：

```
x = df.drop(index=[10, 30])
```

传统轴写法：

```
df.drop(["city"], axis=1)
df.drop([10, 30], axis=0)
```

- axis=0：操作行标签。
- axis=1：操作列标签。

易错点：drop 使用标签，不是行位置。删除第一行可写 `df.drop(index=df.index[0])`。

drop 默认返回新表，原表不变。要更新变量：

```
df = df.drop(columns=["city"])
```

14.7 del 和 pop：直接删除列

```
x = df.copy()
del x["city"]
```

pop 删除并返回该列：

```
x = df.copy()
removed = x.pop("city")
```

removed 是被删除的 Series。

14.8 rename()：修改标签名称

```
x = df.rename(columns={
    "price": "unit_price",
    "qty": "quantity"
})
```

修改行标签：

```
x = df.rename(index={10: "r1", 20: "r2"})
```

rename 是改名字，不是改元素数值。

14.9 replace()：替换数据值

```
x = df.copy()
x["group"] = x["group"].replace({
    "A": "Group A",
    "B": "Group B"
})
```

关键点：rename 改行列标签；replace 改表格里面的值。

15 索引管理：set_index、reset_index、reindex

15.1 set_index()：把一列变成行索引

```
x = df.set_index("name")
```

行标签变成 Apple、Pear、Banana 等。默认 name 不再是普通列。

此时：

```
x.loc["Apple", "price"]
```

得到 10。

保留该列：

```
x = df.set_index("name", drop=False)
```

15.2 reset_index()：恢复默认整数索引

```
x = df.set_index("name")
y = x.reset_index()
```

旧索引 name 恢复成普通列，新行索引为 0、1、2、3、4。

丢弃旧索引：

```
y = x.reset_index(drop=True)
```

易错点：题目要求保留索引时，不要随意 `reset_index`。筛选后出现不连续标签是正常现象，不代表错误。

15.3 reindex()：按指定标签重新排列或补出标签

```
s = pd.Series(
    [1, 7, 3, 9],
    index=["d", "c", "a", "f"]
)

s2 = s.reindex(["a", "b", "c", "d", "e", "f"])
```

结果：

```
a    3.0
b    NaN
c    7.0
d    1.0
e    NaN
f    9.0
```

新标签 b、e 原来没有，所以出现缺失值。

表格同时调整行列：

```
x = df.reindex(
    index=[50, 10, 30],
    columns=["name", "qty", "price"]
)
```

按指定行列顺序返回表格。

关键点：reindex 根据标签重新对齐，不是简单改标签名字。改名字用 rename；改顺序或补指定标签用 reindex。

15.4 整体替换标签

```
x = df.copy()
x.index = ["r1", "r2", "r3", "r4", "r5"]
```

新标签数量必须与行数相同。

Index 本身不能通过单个元素赋值修改：

```
# Do not use:
# x.index[0] = "first"
```

需要整体赋值或使用 rename。

16 排序：按值、按索引、多键排序

16.1 sort_values()：按列值排序

```
x = df.sort_values("price")
```

商品顺序：Kiwi、Apple、Orange、Banana、Pear。

降序：

```
x = df.sort_values("price", ascending=False)
```

16.2 多键排序

解决的问题：先按组别升序，同组内按价格降序。

```
x = df.sort_values(
    by=["group", "price"],
    ascending=[True, False]
)
```

顺序：Banana、Apple、Kiwi、Pear、Orange。

`ascending` 中的布尔值与 by 列表一一对应。

16.3 sort_index()：按标签排序

```
df.sort_index()
df.sort_index(ascending=False)
df.sort_index(axis=1)
```

前两句排序行标签，最后一句按列标签排序列。

16.4 排序后取前几条

```
top3 = df.sort_values("price", ascending=False).head(3)
```

这是先对所有记录排序，再取前三条。

易错点：`df.head(3).sort_values(...)` 只排序原表前三条，通常不是全表前三名。

16.5 nlargest()：取数值最大的若干条

```
top3 = df.nlargest(3, "price")
```

适合简单的数值列前 N 名。有复杂并列排序规则时，用 `sort_values` 明确表达。

17 缺失值、重复值和类型转换

17.1 isna() 与 notna()

```
df["score"].isna()
df["city"].notna()
```

返回布尔 Series。

选择缺失成绩：

```
missing = df.loc[df["score"].isna(), ["name", "score"]]
```

得到 Banana 的记录。

每列缺失数量：

```
df.isna().sum()
```

score、city 各为 1，其他列为 0。

易错点：不要用 `== np.nan` 判断缺失值。

17.2 dropna()：删除缺失记录

```
clean = df.dropna()
```

默认删除含任意缺失值的行，案例中会删除 Banana 和 Kiwi。

只检查成绩列：

```
clean = df.dropna(subset=["score"])
```

只删除 Banana。

删除所有字段都缺失的行：

```
clean = df.dropna(how="all")
```

17.3 fillna()：填充缺失值

```
x = df.copy()
x["score"] = x["score"].fillna(0)
```

用非缺失成绩均值填充：

```
x = df.copy()
x["score"] = x["score"].fillna(x["score"].mean())
```

案例中均值为 81.25。

关键点：填 0、填均值、删除记录代表不同处理假设，应按题目和数据含义选择，不存在一律正确的方案。

Pandas 的常见统计默认跳过缺失：

```
df["score"].mean()
```

得到 81.25。这与 NumPy 普通 mean 遇 NaN 返回 NaN 的行为不同。

17.4 duplicated() 与 drop_duplicates()

```
orders = pd.DataFrame({
    "id": [1, 1, 2],
    "name": ["Tom", "Tom", "Amy"]
})

orders.duplicated()
```

默认把第一次之后的重复标记为 True。

删除完全重复行：

```
clean = orders.drop_duplicates()
```

按指定字段去重：

```
clean = orders.drop_duplicates(subset=["id"], keep="first")
```

keep 可以选 first、last，或 False。False 表示重复组中的所有记录都删除。

17.5 astype() 与 to_numeric()

```
x = pd.Series(["10", "20", "30"])
numbers = x.astype(int)
```

如果包含错误字符串：

```
x = pd.Series(["10", "bad", "30"])
numbers = pd.to_numeric(x, errors="coerce")
```

结果：

```
0    10.0
1     NaN
2    30.0
dtype: float64
```

coerce 把无法转换的值设为缺失。

易错点：含 NaN 的数据不能直接转成普通 int。可以先处理缺失，或按需要使用可空整数类型 `astype("Int64")`。

18 copy：保留原表，避免链式赋值

18.1 赋值不是复制

```
x = df
```

两个变量指向同一个 DataFrame 对象。对 x 的直接修改也可能改变 df。

18.2 copy() 创建独立表格副本

```
x = df.copy()
x.loc[10, "price"] = 99
```

对本讲义这些普通数值与字符串数据，df 的 price 不会因此改变。

Pandas 的 copy 与 NumPy 的视图规则不能简单一一套用；不同 Pandas 版本还可能使用 Copy-on-Write。考试写法应优先清楚表达是否需要修改原表。

18.3 筛选后要修改，显式 copy

```
selected = df.loc[df["qty"] >= 3].copy()
selected["revenue"] = selected["price"] * selected["qty"]
```

这样明确创建供后续修改的独立结果。

关键点：修改原表用单次 loc 赋值；建立独立分析表用 copy。不要依赖链式赋值的隐含行为。

19 groupby：分组统计

19.1 groupby 到底做什么

解决的问题：分别计算 A 组和 B 组商品的总数量。

groupby 的思路：

1. 按组别把记录分开。
2. 对每组计算统计量。
3. 把结果合起来。

```
result = df.groupby("group")["qty"].sum()
```

```
print(result)
```

输出：

```
group
A    11
B     4
Name: qty, dtype: int64
```

A 组数量为 $2+4+5=11$，B 组为 $1+3=4$。

19.2 分组均值

```
df.groupby("group")["price"].mean()
```

结果：

```
group
A    11.0
B    16.0
```

先按 group 分组，再选择 price，最后 mean。

19.3 as_index=False：组别保持普通列

```
result = df.groupby(
    "group",
    as_index=False
)["qty"].sum()
```

输出：

```
  group  qty
0     A   11
1     B    4
```

默认 groupby 常把组别作为结果索引。`as_index=False` 让它成为普通列，返回表格更方便。

19.4 agg()：一次计算多个统计量

对一列计算多个统计：

```
result = df.groupby("group")["price"].agg(
    ["mean", "min", "max"]
)
```

结果：

```
       mean  min  max
group
A      11.0    8   15
B      16.0   12   20
```

19.5 命名聚合：直接指定输出列名

```
result = df.groupby("group", as_index=False).agg(
    mean_price=("price", "mean"),
    total_qty=("qty", "sum"),
    valid_score_count=("score", "count")
)
```

每个输出字段的结构是：

```
output_name=("source_column", "operation")
```

案例结果：

```
  group  mean_price  total_qty  valid_score_count
0     A        11.0         11                  2
1     B        16.0          4                  2
```

19.6 size() 与 count() 的差别

```
df.groupby("group").size()
df.groupby("group")["score"].count()
```

第一项统计记录数：A 为 3，B 为 2。

第二项统计非缺失成绩数：A 为 2，B 为 2。

关键点：组内记录数用 size，某字段非缺失数量用 count，销量总量用 sum。不要混用。

19.7 多个字段分组

```
result = df.groupby(
    ["group", "city"],
    dropna=False
)["qty"].sum()
```

按组别和城市的组合分组。默认 groupby 通常不把缺失分组键形成普通组；需要包含缺失键时可显式使用 dropna=False。

19.8 transform()：把组统计结果放回每一行

解决的问题：给每个商品附上它所在组的平均价格。

```
x = df.copy()
x["group_mean_price"] = (
    x.groupby("group")["price"].transform("mean")
)
```

新列：

```
[11.0, 16.0, 11.0, 16.0, 11.0]
```

普通聚合每组一个结果；transform 返回与原记录对应的结果。

可以筛选高于本组均价的商品：

```
selected = x[
    x["price"] > x["group_mean_price"]
]
```

选到 Pear、Banana。

20 cut：把连续数值分成类别

20.1 分箱是什么

把具体年龄转换为“少年、青年、中年、老年”，就是把连续数值划分到区间中。

```
users = pd.DataFrame({
    "name": ["Tom", "Amy", "Bob", "Eve", "Leo"],
    "age": [12, 18, 35, 60, 80],
    "sex": ["M", "F", "M", "F", "M"]
})
```

20.2 pd.cut()：指定边界

```
users["age_group"] = pd.cut(
    users["age"],
    bins=[0, 18, 40, 60, np.inf],
    labels=["Child", "Young", "Middle", "Senior"],
    right=False
)
```

right=False 表示左闭右开：

- Child：$[0, 18)$。

- Young：$[18, 40)$。
- Middle：$[40, 60)$。
- Senior：$[60, +\infty)$。

结果：

```
Tom -> Child
Amy -> Young
Bob -> Young
Eve -> Senior
Leo -> Senior
```

关键点：bins 有 5 个边界，就形成 4 个区间，labels 必须有 4 个标签。边界归属必须仔细检查，尤其是 18、40、60。

20.3 默认 right=True

默认区间通常是：

$$(0, 18],\ (18, 40],\ (40, 60],\ (60, +\infty].$$

若希望最左端也被纳入，可以使用 `include_lowest=True`。

超出 bins 覆盖范围的值会被分成缺失值，不会自动塞入最边缘类别。

20.4 统计各年龄组数量

```
counts = users["age_group"].value_counts(sort=False)
```

结果：Child 1，Young 2，Middle 0，Senior 2。

20.5 年龄组中的男女数量与比例

```
table = pd.crosstab(
    users["age_group"],
    users["sex"]
)
```

按行计算比例：

```
ratios = pd.crosstab(
    users["age_group"],
    users["sex"],
    normalize="index"
)
```

normalize=”index” 表示每个年龄组内部比例合计为 1。

20.6 qcut()：按分位数分组

```
x = pd.Series([10, 20, 30, 40, 50, 60])

groups = pd.qcut(
    x,
    q=3,
    labels=["Low", "Middle", "High"]
)
```

cut 是按数值边界分区间；qcut 是根据数据分位数，尽量让各组人数接近。

重复值较多时，分位边界可能重复，需要按题目考虑 duplicates 等参数。不要把 cut 和 qcut 当成同一件事。

21 merge：连接两张表

21.1 为什么要合并表格

一张投资表存公司编号和投资额，另一张公司表存公司编号和名称。分析时需要通过公司编号把名称补过来。

```
investment = pd.DataFrame({
    "firm_id": ["F1", "F2", "F1", "F3"],
    "year": [2020, 2020, 2021, 2021],
    "invest": [100, 80, 120, 60]
})

firms = pd.DataFrame({
    "firm_id": ["F1", "F2"],
    "firm": ["Alpha", "Beta"]
})
```

21.2 left merge：保留左表全部记录

```
merged = investment.merge(
    firms,
    on="firm_id",
    how="left"
)
```

结果：

```
  firm_id  year  invest   firm
0      F1  2020     100  Alpha
```

```
1      F2  2020      80   Beta
2      F1  2021     120  Alpha
3      F3  2021      60    NaN
```

F3 没有名称匹配，所以出现缺失。

- on：连接键。
- how：保留哪些记录。

21.3 常见连接方式

how	含义
inner	只保留双方都匹配的键
left	保留左表所有记录
right	保留右表所有记录
outer	保留两表所有键

inner 连接：

```
investment.merge(firms, on="firm_id", how="inner")
```

F3 的记录不在结果中。

21.4 两边连接列名称不同

```
firms2 = firms.rename(columns={"firm_id": "id"})

merged = investment.merge(
    firms2,
    left_on="firm_id",
    right_on="id",
    how="left"
)
```

21.5 多个连接键

```
result = left_table.merge(
    right_table,
    on=["firm_id", "year"],
    how="left"
)
```

必须两个字段同时匹配。

21.6 为什么合并后行数会增加

若右表同一个连接键出现多次，左表对应记录会匹配多条右表记录。多对多匹配可能让行数大幅增加。

如果右表公司编号应唯一，可验证：

```
merged = investment.merge(
    firms,
    on="firm_id",
    how="left",
    validate="many_to_one"
)
```

`many_to_one` 表示左边可以重复，右边的连接键必须唯一。

关键点：合并后检查行数、缺失和连接键唯一性。“代码能运行”不保证连接关系符合题意。

22 concat：把表格上下或左右拼接

22.1 上下追加记录

```
a = pd.DataFrame({
    "name": ["Tom", "Amy"],
    "score": [80, 90]
})

b = pd.DataFrame({
    "name": ["Bob"],
    "score": [55]
})

combined = pd.concat([a, b], ignore_index=True)
```

结果：

```
  name  score
0  Tom     80
1  Amy     90
2  Bob     55
```

`ignore_index=True` 重新生成连续行索引。默认保留原标签，可能出现重复标签。

22.2 左右拼接列

```
left = pd.DataFrame({
    "name": ["Tom", "Amy"]
}, index=[10, 20])

right = pd.DataFrame({
    "score": [90, 80]
}, index=[20, 10])

combined = pd.concat([left, right], axis=1)
```

按索引对齐，所以标签 10 对应 Tom、80，标签 20 对应 Amy、90。

关键点：concat 左右拼接主要按索引对齐；merge 是按指定连接键匹配。两者不能随意互换。

23 透视表、交叉表与长宽表转换

23.1 pivot_table()：二维分组汇总

```
sales = pd.DataFrame({
    "store": ["A", "A", "A", "B", "B"],
    "product": ["Apple", "Apple", "Pear", "Apple", "Pear"],
    "qty": [2, 3, 4, 1, 6]
})
```

想得到“门店为行、商品为列、销量为值”的汇总表：

```
table = sales.pivot_table(
    index="store",
    columns="product",
    values="qty",
    aggfunc="sum",
    fill_value=0
)
```

结果：

```
product  Apple  Pear
store
A            5     4
B            1     6
```

参数：

- index：结果的行分类。
- columns：结果的列分类。

- values：要统计的数值列。
- aggfunc：统计方法。
- `fill_value`：汇总后缺失单元格的填充值。

易错点：`pivot_table` 默认聚合通常是 mean。题目要求总量时，要写 aggfunc="sum"。

23.2 pivot()：只重排，不聚合

```
wide = pd.DataFrame({
    "store": ["A", "A", "B", "B"],
    "product": ["Apple", "Pear", "Apple", "Pear"],
    "qty": [5, 4, 1, 6]
})

table = wide.pivot(
    index="store",
    columns="product",
    values="qty"
)
```

同一行分类与列分类的组合必须唯一。如果 A、Apple 有两条记录，pivot 无法决定放哪一个；需要聚合时用 `pivot_table`。

23.3 crosstab()：分类交叉计数

```
users = pd.DataFrame({
    "group": ["A", "A", "B", "B", "B"],
    "sex": ["M", "F", "M", "M", "F"]
})

table = pd.crosstab(users["group"], users["sex"])
```

结果：

```
sex    F  M
group
A      1  1
B      1  2
```

不提供数值列时，主要统计类别组合出现次数。

23.4 melt()：宽表变成长表

```
scores = pd.DataFrame({
```

```
    "name": ["Tom", "Amy"],
    "math": [80, 90],
    "english": [70, 95]
})

long = scores.melt(
    id_vars="name",
    value_vars=["math", "english"],
    var_name="subject",
    value_name="score"
)
```

结果：

```
  name  subject  score
0  Tom     math     80
1  Amy     math     90
2  Tom  english     70
3  Amy  english     95
```

- `id_vars`：保持身份的列。
- `value_vars`：要展开的列。
- `var_name`：原列名放入的新列名称。
- `value_name`：原数值放入的新列名称。

长表每行记录一个“人、科目、成绩”的组合，方便继续分组分析。

23.5 stack() 与 unstack()

```
summary = sales.groupby(["store", "product"])["qty"].sum()
table = summary.unstack(fill_value=0)
```

unstack 把某层行索引展开成列。stack 则把列中的某一层转到行索引。

这两个操作与多层索引有关。初学时先掌握 `pivot_table` 和 melt 的具体含义，再按题目需要使用 stack、unstack。

24 综合案例一：销售分析流水线

24.1 题意

给定销售记录：

1. 只保留数量至少 3 的记录。

2. 计算销售额 revenue。
3. 保留销售额至少 40 的记录。
4. 只输出 name、group、revenue。
5. 按 revenue 降序；并列时 name 升序。
6. 取前两条，保存到 result。
7. 不修改原表。

24.2 分步答案

先创建案例表，见第 5 节。

```
selected = df.loc[df["qty"] >= 3].copy()

selected["revenue"] = (
    selected["price"] * selected["qty"]
)

selected = selected.loc[
    selected["revenue"] >= 40,
    ["name", "group", "revenue"]
]

result = selected.sort_values(
    by=["revenue", "name"],
    ascending=[False, True]
).head(2)
```

结果：

```
      name group  revenue
30  Banana     A       60
50    Kiwi     A       40
```

24.3 为什么这个顺序重要

- copy 明确不修改原表。
- 先新增 revenue，才能按它筛选。
- 先筛选，再排序，再 head。
- 输出列顺序按题目要求明确写出。
- 不重置索引，因此保留标签 30、50。

关键点：先 head 再筛选或排序，可能把后面真正符合条件的记录提前丢掉。

25 综合案例二：公司投资统计与合并

25.1 准备数据

```
investment = pd.DataFrame({
    "firm_id": ["F1", "F2", "F1", "F2", "F3"],
    "year": [2020, 2020, 2021, 2021, 2021],
    "invest": [100, 80, 120, 90, 60],
    "capital": [90, 100, 100, 70, 50]
})

firms = pd.DataFrame({
    "firm_id": ["F1", "F2", "F3"],
    "firm": ["Alpha", "Beta", "Gamma"]
})
```

25.2 任务

只分析 invest 大于 capital 的记录。按公司统计投资总额与记录数，补充公司名称，最后按投资总额降序排列。

```
selected = investment.loc[
    investment["invest"] > investment["capital"]
].copy()

summary = selected.groupby(
    "firm_id",
    as_index=False
).agg(
    total_invest=("invest", "sum"),
    record_count=("invest", "size")
)

result = summary.merge(
    firms,
    on="firm_id",
    how="left",
    validate="many_to_one"
)

result = result[
    ["firm_id", "firm", "total_invest", "record_count"]
].sort_values(
```

```
    ["total_invest", "firm_id"],
    ascending=[False, True]
)
```

结果：

```
  firm_id   firm  total_invest  record_count
0      F1  Alpha           220             2
1      F2   Beta            90             1
2      F3  Gamma            60             1
```

关键点：筛选前 F2 有两条投资记录，但其中一条不满足 invest 大于 capital。必须先筛选再分组，因此 F2 的投资总额是 90，记录数是 1。

26 按需求查找：常用函数速查

需求	常用写法
读取与保存	`pd.read_csv(...); df.to_csv(..., index=False)`
预览与结构	`df.head(); df.info(); df.shape`
单列与多列	`df["price"]; df[["price", "qty"]]`
标签与位置	`df.loc[20, "price"]; df.iloc[1, 2]`
条件筛选	`df.loc[df["qty"] >= 3]`
多条件筛选	`df.loc[(df["price"] >= 10) & (df["qty"] >= 3)]`
字符串匹配	`df["name"].str.contains("a", case=False, na=False)`
新增列	`df["revenue"] = df["price"] * df["qty"]`
修改原表	`df.loc[df["group"] == "B", "price"] = 99`
删除列	`df.drop(columns=["city"])`
排序	`df.sort_values("price", ascending=False)`
索引管理	`df.set_index("name"); df.reset_index()`
缺失检查	`df.isna().sum()`
填补与删除	`df.fillna({"score": 0}); df.dropna(subset=["score"])`
重复检查	`df.duplicated(); df.drop_duplicates()`
数值转换	`pd.to_numeric(df["price"], errors="coerce")`
不同值频数	`df["group"].value_counts(dropna=False)`
分组总和	`df.groupby("group")["qty"].sum()`
分组记录数	`df.groupby("group").size()`

需求	常用写法
组内广播	`df.groupby("group")["qty"].transform("sum")`
分箱	`pd.cut(df["price"], bins=[0, 10, 20])`
按键合并	`left.merge(right, on="id", how="left")`
上下拼接	`pd.concat([a, b], ignore_index=True)`
透视汇总	`df.pivot_table(index="group", values="qty", aggfunc="sum")`
长宽转换	`df.melt(id_vars=["name"], value_vars=["price", "qty"])`

27 综合练习与参考答案

以下练习均从第5节的原始 df 开始，各题独立运行。先自己写代码，再对照答案；不要沿用前一题修改后的表格。

27.1 练习一：标签、位置和输出类型

取标签为 20、50 的商品名称与单价，按指定顺序返回 DataFrame。再分别取第二行的名称和仅含第二行的表格。

```
answer = df.loc[[20, 50], ["name", "price"]]
name = df.iloc[1, 0]
row = df.iloc[[1]]
```

answer 的两行分别是 Pear（20）和 Kiwi（8）；name 是字符串 Pear，row 是一行七列的 DataFrame。

27.2 练习二：多条件筛选与排序

选择 A 组且数量至少为 3 的商品，计算销售额，按销售额降序输出名称和销售额。

```
x = df.loc[(df["group"] == "A") & (df["qty"] >= 3)].copy()
x["revenue"] = x["price"] * x["qty"]
answer = x.sort_values("revenue", ascending=False)[["name", "revenue"]]
```

结果为 Banana：60；Kiwi：40。保留行标签 30、50。

27.3 练习三：缺失值与组内填补

统计 score 缺失数量，再用各组非缺失 score 的均值填补，并保留原表。

```
missing_count = df["score"].isna().sum()
```

```
x = df.copy()
group_mean = x.groupby("group")["score"].transform("mean")
x["score"] = x["score"].fillna(group_mean)
```

缺失数量是 1；Banana 的 score 填成 A 组均值 82.5。如果某组全是缺失，组均值也是缺失，仍需另行制定填补规则。

27.4 练习四：分组汇总

每组统计总数量、平均单价、记录数和有效 score 数量。

```
answer = df.groupby("group", as_index=False).agg(
    total_qty=("qty", "sum"),
    mean_price=("price", "mean"),
    rows=("score", "size"),
    valid_scores=("score", "count")
)
```

A 组依次为 11、11、3、2；B 组依次为 4、16、2、2。

关键点：size 统计组内行数；count 统计指定列的非缺失值数。

27.5 练习五：分箱边界

按单价分为低价与高价两档，区间分别为 [0, 10] 和 (10, 20]。

```
band = pd.cut(
    df["price"], bins=[0, 10, 20],
    labels=["low", "high"], right=True, include_lowest=True
)
```

按原行顺序得到 low、high、high、high、low。单价 10 属于低价，20 属于高价；超出边界的值会成为缺失值。

27.6 练习六：合并并核对匹配情况

给商品表补充每组的负责人，并验证组信息表中每组只有一条记录。

```
owners = pd.DataFrame({"group": ["A", "B"], "owner": ["Alice", "Bob"]})
answer = df.merge(
    owners, on="group", how="left",
    validate="many_to_one", indicator=True
)
assert len(answer) == len(df)
assert answer["_merge"].eq("both").all()
```

A 组负责人是 Alice，B 组是 Bob。按列合并会生成新的行索引；若题目需要保留原行标签，应在合并前把索引保存成列，合并后再恢复。

28 做题前的最后检查

1. 确认行标签与位置，明确需要 Series、DataFrame 还是单个值。
2. 写清筛选、计算、分组、合并、排序、截取的先后顺序。
3. 检查缺失处理规则，以及 size、count、sum 的统计含义。
4. 明确修改原表还是创建副本；修改原表使用单次 loc 赋值。
5. 核对合并键、匹配情况、结果行数、列顺序和索引要求。